A Tiny Media Server in Rust

Notes for a C++ Programmer Who Has Never Seen Rust

Contents

1 What we built **1**

2 Rust from the ground up (for C++ people) **2**

2.1 Tooling . . . 2
2.2 Basic syntax . . . 2
2.3 Ownership and borrowing . . . 2
2.4 Lifetimes and `'static` . . . 3
2.5 Errors: `Result`, `?`, `Option` . . . 3
2.6 Structs, modules, macros . . . 3
2.7 Closures and threads . . . 3

3 Walking through `main.rs` **3**

3.1 Constants and embedding the file . . . 3
3.2 Building one buffer at startup . . . 4
3.3 The accept loop . . . 4
3.4 Reading the request . . . 4
3.5 Range requests . . . 4
3.6 Why drain before closing? . . . 4

4 Cargo.toml and the build **5**

5 Verifying it **5**

6 Security review **5**

7 Limitations and exercises **6**

1 What we built

A program that listens on a TCP port and answers every connection with one audio (or video) file baked into the executable. Open `http://localhost:3005` in a browser and it plays.

- `src/main.rs` – the whole server (about 80 lines).
- `media/3005.bbd` – the file that gets embedded. Replace it and rebuild.
- `Cargo.toml` – project and optimisation settings.

- `.cargo/config.toml` – makes the default build target static Linux (musl).

Build and run:

```
cargo build --release
./target/x86_64-unknown-linux-musl/release/mediaserver
```

2 Rust from the ground up (for C++ people)

2.1 Tooling

`cargo` is CMake + a package manager + a test runner in one. `rustc` is the compiler (like `g++`); you almost never call it directly. `cargo build --release` is an optimised build; plain `cargo build` is a debug build.

2.2 Basic syntax

```
fn add(a: i32, b: i32) -> i32 { // types come after names
    a + b // no semicolon = this is the return value
}
let x = 5; // immutable by default (like const auto)
let mut y = 5; // mutable
const PORT: u16 = 3005; // compile-time constant, type required
```

Integer types are explicit: `u8, u16, u32, u64, usize` (unsigned; `usize` is pointer-sized, like `size_t`) and `i8..i64, isize` (signed). Almost everything is an expression: `if`, `match` and blocks all produce values.

2.3 Ownership and borrowing

This is the big difference from C++. Every value has exactly one owner; when the owner goes out of scope the value is dropped (RAII, always). Assigning or passing a value *moves* it – the old name becomes unusable (like `std::move`, but enforced by the compiler). Instead of moving you can *borrow*:

Rust	Roughly C++
`&T`	`const T&` (many allowed at once)
`&mut T`	`T&` (exactly one, and no `&T` at the same time)
`Vec<u8>`	`std::vector<uint8_t>`
`&[u8]`	`std::span<const uint8_t>` (pointer + length, a "slice")
`String` / `&str`	`std::string` / `std::string_view`
`Box<T>`	`std::unique_ptr<T>`
`Option<T>`	`std::optional<T>`
`Result<T,E>`	value-or-error (no exceptions in Rust)

The compiler proves at build time that no reference outlives what it points to and that you never have a data race. That removes dangling pointers and use-after-free in safe code.

2.4 Lifetimes and `'static`

A reference type can carry a lifetime: `&'a [u8]`. `&'static` means "valid for the whole program". String literals and `include_bytes!` data are `'static` because they live in the executable's read-only data.

2.5 Errors: `Result`, `?`, `Option`

```
let n = s.read(&mut buf)?; // if Err, return it from this function immediately
let v = x.expect("msg"); // if Err/None, crash with msg (fine at startup)
let _ = might_fail(); // deliberately ignore a result
```

Ignoring a `Result` without `let _ =` produces a compiler warning – Rust makes you say that you meant it.

2.6 Structs, modules, macros

`struct Resp { ... }` is a plain data type (no inheritance). Names ending in `!` (`include_bytes!`, `format!`, `macro_rules!`) are macros, expanded at compile time – hygienic, and far saner than the C preprocessor. `use std::io::Read;` is like `using`/`#include`: it brings a name (here a *trait*, similar to an interface) into scope so its methods can be called.

2.7 Closures and threads

`|a, b| a + b` is a lambda. `move || ...` captures by value, i.e. the closure takes ownership of what it uses. `std::thread::spawn(closure)` runs it on a new OS thread. The compiler only allows data to cross into a thread if it is safe to do so – here, either owned by the closure or `'static`.

3 Walking through `main.rs`

3.1 Constants and embedding the file

The port is hardcoded to 3005. The line `const _: () = assert!(PORT == 0x0BBD, ...)` is evaluated by the compiler: `0x0BBD` is 3005 in hexadecimal ($11 \cdot 256 + 11 \cdot 16 + 13$), so editing `PORT` to any other value is a *build error*, not a runtime one. (`const _` is an unnamed constant whose only job is to run the check, similar to a C++ `static_assert`.)

```
const PORT: u16 = 3005;
macro_rules! media_path { () => { "../media/3005.bbd" }; }
const PATH: &str = media_path!();
static MEDIA: &[u8] = include_bytes!(media_path!());
```

`include_bytes!` reads the file at *compile time* and puts its bytes in the executable (think `xxd -i`, built in). Its argument must be a string literal, not a `const`, so a tiny macro supplies the path in one place; the same text is reused as `PATH` to choose the MIME type. The path is relative to the source file (`src/`), hence `../media`.

3.2 Building one buffer at startup

We format the header once, append the body, then `Box::leak` the buffer. `Box::leak` deliberately never frees the memory and hands back a `&'static mut`; we use it as an immutable `&'static [u8]`. Leaking is fine because the data should live as long as the process, and being `'static` lets every thread share it with no locks and no reference counting.

The header is:

```
HTTP/1.1 200 OK\r\n
Content-Type: audio/mpeg\r\n
Content-Length: <n>\r\n
Accept-Ranges: bytes\r\n
Connection: close\r\n
\r\n
```

HTTP headers end with an empty line (`\r\n\r\n`). `Content-Length` tells the browser when the body ends; `Connection: close` says we won't reuse the socket. `Accept-Ranges` advertises seeking support (an addition to the minimum list; Safari likes it).

3.3 The accept loop

```
let listener = TcpListener::bind(("0.0.0.0", PORT)).expect("bind");
for stream in listener.incoming().flatten() {
    std::thread::spawn(move || { let _ = handle(stream, resp); });
}
```

`TcpListener` wraps `socket/bind/listen/accept`. `incoming()` yields a `Result<TcpStream>` for each connection; `.flatten()` drops the failures. Each accepted stream is *moved* into a fresh thread. When `handle` returns, the stream is dropped, which closes the socket (RAII).

3.4 Reading the request

We loop on `read` until the bytes seen so far contain `\r\n\r\n` (capped at 64 KiB so a rude client can't make us buffer forever). We never parse the method or path – every URL gets the same file.

3.5 Range requests

When you seek, or when Safari/iOS opens media, the browser sends `Range: bytes=100-199`. We answer `206 Partial Content` with `Content-Range: bytes 100-199/total` and only those bytes. `parse_range` accepts a single `bytes=a-b` or `bytes=a-`; anything else (multiple ranges, suffix form `-500`, garbage, out-of-bounds start) returns `None` and we fall back to the full 200. The end is clamped to the last byte. A 206 header depends on the request so it is formatted per request (the 200 path remains one pre-built `write_all`); the body slice is zero-copy: `&body[a..=b]`.

3.6 Why drain before closing?

If a program closes a TCP socket while unread data is still in its receive buffer, Linux sends a *RST* (reset) instead of a normal *FIN*. A RST can make the client discard response bytes that were still in flight, truncating the file. So after writing we call `shutdown(Write)` (sends FIN – "I'm done sending") and then read until the client closes its side, and only then drop the socket.

4 Cargo.toml and the build

Setting	Effect
`opt-level = 3`	like `-O3`
`lto = "fat"`	whole-program link-time optimisation
`codegen-units = 1`	one unit: slower compile, better optimisation
`panic = "abort"`	a panic kills the process (no unwinding code); a panic in one connection thread therefore takes down the whole server
`strip = true`	remove symbols; smaller binary

The target `x86_64-unknown-linux-musl` links against musl libc statically, so the binary has no dependencies and runs on any x86-64 Linux. One-time setup: `rustup target add x86_64-unknown-linux-musl`. No `libc` crate or `sendfile` was needed; with the file in memory, `write_all` is already a straight copy to the kernel.

5 Verifying it

```
curl -I localhost:3005 # headers
curl -r 0-99 -o part.bin localhost:3005 # 206, exactly 100 bytes
curl -o all.bin localhost:3005
cmp all.bin media/3005.bbd # no output = identical
```

`curl -I` sends a HEAD request; because we do not parse methods the server also sends the body, which curl ignores.

6 Security review

Threat model: anyone who can reach port 3005 sends arbitrary bytes. Goal: they can't run code, escalate, or kill the server.

- **No memory-unsafe code.** `#![forbid(unsafe_code)]` makes the compiler reject any `unsafe` block, and there are no dependencies, so buffer overflows and use-after-free are not possible in this program.
- **Request is never interpreted.** Only the `Range` header is read, and its numbers are parsed with overflow-checked `parse` and clamped to the file length. No input is ever echoed into a response (no header injection), used as a file path (no traversal), or passed to a shell. The audio is embedded, so the server never opens files at runtime.
- **Panic = total outage.** With `panic = "abort"` any panic kills the whole server. Two panic sources were removed: slicing a `&str` at a non-character boundary (a `Range:` line containing multibyte UTF-8 would have crashed the process, so parsing is now done on raw bytes) and `thread::spawn` failing (now `Builder::spawn`, whose error is handled).
- **Resource limits.** Max 256 simultaneous connections (extras are closed immediately), 16 KiB request cap, 10 s read/write timeouts, bounded post-response drain, 64 KiB thread stacks, and a header scan that is linear (not quadratic) in slow-drip input.

- **Privilege.** The server refuses to start as root (it checks the effective uid in `/proc/self/status`). Port 3005 is above 1024, so root is never needed. Run it as an unprivileged user, ideally under systemd with `DynamicUser=yes`, `NoNewPrivileges=yes`, `ProtectSystem=strict`, `ProtectHome=yes`, `PrivateTmp=yes`. The static binary needs no files at all.
- **Audio-specific.** The server treats the file as opaque bytes and never decodes it, so a malformed MP3 cannot exploit the server. It could only target the *listener's* decoder, so embed only audio you trust. Responses carry `X-Content-Type-Options: nosniff` (the browser must treat the bytes as audio, never sniff HTML/JS out of them). I deliberately do *not* send `Content-Security-Policy: sandbox`: it looks protective but breaks the browser's built-in media player (stuck at 0:00).
- **Not covered.** No TLS (traffic is plaintext, visible and modifiable on the network), and bandwidth floods must be handled upstream. No software can honestly be called "provably" secure; this is a small, dependency-free design reviewed and tested against the cases above.

7 Limitations and exercises

- A client that reads very slowly can hold one of the 256 connection slots for a long time (each individual read/write has a 10 s timeout, but a whole transfer does not). Put a reverse proxy or firewall rate limit in front for hostile networks.
- No HEAD, 404, 416 handling – by design.
- Exercise: support the suffix form `bytes=-500`.
- Exercise: switch to `.mp4` (change the macro, drop in a file) and check the `Content-Type`.